Mini Project Report

on

Design and Development of a Lightweight Question-Generation LLM from Scratch

Submitted by

Kamal Nayan Kumar (23BDS026)

Altaf Raja (23BCS011)

Rahul Patel (23BDS047)

Vijaypal Singh Rathore (23BDS067)

Under the guidance of

Dr. Krishnendu Ghosh

Project Supervisor

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

INDIAN INSTITUTE OF INFORMATION TECHNOLOGY DHARWAD

Academic Year 2025–26

April 2026

Certificate

This is to certify that the project entitled **Design and Development of a Lightweight Question-Generation LLM from Scratch** is a bonafide record of the Mini Project coursework presented by the students whose names are given below during the academic year **2025–26** in partial fulfilment of the requirements of the degree of Bachelor of Technology in Computer Science and Engineering.

Roll No.	**Name of Student**
23BDS026	Kamal Nayan Kumar
23BCS011	Altaf Raja
23BDS047	Rahul Patel
23BDS067	Vijaypal Singh Rathore

Dr. Krishnendu Ghosh
(Project Supervisor)

Contents

List of Figures iii

List of Tables iv

Abstract v

1 Introduction 1
- 1.1 Problem Statement . . . 2
- 1.2 Project Contributions . . . 2

2 Related Work 3
- 2.1 Transformer Language Models . . . 3
- 2.2 Architectural Contributions . . . 3
- 2.3 Automatic Question Generation . . . 4
- 2.4 Evaluation Metrics for Text Generation . . . 5

3 Data and Methods 5
- 3.1 End-to-End Pipeline Overview . . . 5
- 3.2 Pretraining Datasets . . . 6
- 3.3 Preprocessing Pipeline . . . 7
- 3.4 Tokenization . . . 7
- 3.5 Model Architecture . . . 8
- 3.6 Supervised Fine-Tuning Dataset . . . 9
- 3.7 Training Methodology . . . 10
  - 3.7.1 Pretraining . . . 10
  - 3.7.2 Supervised Fine-Tuning . . . 12
- 3.8 Inference Engine . . . 12

4 Results and Discussion 13
- 4.1 Pretraining Learning Curve . . . 13
- 4.2 Supervised Fine-Tuning Results . . . 14
- 4.3 Quantitative Evaluation . . . 14
- 4.4 Analysis of Evaluation Metrics . . . 15
- 4.5 Inference Demonstrations . . . 16
- 4.6 Parameter Efficiency Analysis . . . 18
- 4.7 Training Compute and Efficiency . . . 18
- 4.8 Key Innovations . . . 19

5 Conclusion 19

6 Project Resources and Links **21**
6.1 GitHub Repository 21
6.2 HuggingFace Model Hub 21
6.3 HuggingFace Datasets 21
6.4 Key Source Code Files 22
6.5 Hardware and Platform Details 23

References **23**

List of Figures

1 End-to-end training pipeline. 6
2 Pretraining corpus and SFT dataset composition. 7
3 Model architecture diagram. 8
4 Pretraining validation loss curve. 13
5 SFT validation loss and perplexity per epoch. 14
6 Evaluation metrics: bar chart and radar chart. 15
7 Demo 1: Short-answer question (biology). 16
8 Demo 2: Short-answer question (physics). 17
9 Demo 3: Complex question (history). 17
10 Parameter efficiency and architecture comparison. 18
11 Training compute time and learning progress. 18

List of Tables

1 Pretraining corpus sources and mixing ratios 6
2 Complete model configuration 9
3 SFT source files and record counts 10
4 Pretraining hyperparameters 11
5 Supervised fine-tuning hyperparameters 12
6 SFT validation response loss and perplexity per epoch 14
7 Quantitative evaluation results (50 examples per benchmark) 15
8 Key source code modules and their purpose 22
9 Computing infrastructure used in each training phase 23

Abstract

This mini project presents the complete design, training, and evaluation of a lightweight, decoder-only Large Language Model (LLM) built entirely from scratch for educational question generation. Rather than adapting a large pre-trained commercial model through prompt engineering, the work covers the entire lifecycle of modern LLM development: corpus collection and cleaning, tokenization, autoregressive pretraining, instruction-style supervised fine-tuning (SFT), inference, and evaluation.

The resulting model, referred to as the SLM (Small Language Model), contains approximately 114.1 million trainable parameters. It adopts several modern architectural choices drawn from state-of-the-art research: Grouped Query Attention (GQA) for memory-efficient attention, Rotary Positional Embeddings (RoPE) for relative position encoding, RMSNorm for stable pre-normalization, SwiGLU feed-forward blocks for improved expressiveness, and weight tying between the token embedding and the language-model head. The model operates over a 4096-token context window and uses the `tiktoken r50k_base` vocabulary extended with three special instruction tokens.

Pretraining was performed on approximately 3.0 billion tokens drawn from a carefully mixed corpus of Wikipedia articles, OpenWebText, Project Gutenberg books, and Medium articles, using an NVIDIA H100 GPU on the Lightning AI platform. The resulting base model achieved a validation loss of approximately 2.71 on the pretraining corpus. Supervised fine-tuning on 88,275 training samples drawn from SQuAD v2 and HotpotQA datasets then specialized the model for two distinct question formats: short-answer and complex multi-hop reasoning questions.

Evaluation on 50 held-out examples from SQuAD v2 (short) and HotpotQA (complex) shows BERTScore F1 values of 0.8945 and 0.8643, respectively. These scores confirm that the model generates semantically accurate, contextually relevant questions despite its compact size. The near-zero BLEU-4 score is expected for generative question-generation tasks, where lexical overlap is a poor proxy for question quality. The project demonstrates that a carefully designed 114M-parameter model trained from scratch with modern architectural choices can serve as a practical educational AI tool without relying on expensive cloud inference APIs.

1 Introduction

Automatic question generation (AQG) is the task of producing meaningful, grammatically correct, and educationally relevant questions from a given passage of text. It is a foundational capability in educational technology, enabling a wide range of downstream applications such as adaptive quiz creation, intelligent tutoring systems, reading comprehension assessment, and self-study tools. Traditionally, preparing high-quality assessment questions is a manual, time-consuming process that requires domain expertise. As the scale and diversity of educational content grow rapidly-particularly in online and blended learning environments-automated approaches to question generation become increasingly important.

Recent advances in large-scale transformer-based language models have shown that autoregressive decoder-only architectures can learn powerful generative behaviour from next-token prediction over large corpora [11, 17]. However, the largest and most capable models-such as GPT-4 and LLaMA-2 [16]-contain hundreds of billions of parameters and require enormous computational resources for both training and inference. Such systems are impractical for academics, resource-constrained institutional deployments, and privacy-sensitive educational environments where data cannot be sent to external APIs.

This project investigates an alternative strategy: designing and training a compact but architecturally modern decoder-only transformer entirely from scratch, targeting the educational question-generation task specifically. Rather than borrowing pretrained weights from a large model and applying parameter-efficient fine-tuning *on top*, the project starts from randomly initialized weights and builds up competence through two sequential training stages: autoregressive pretraining on a large mixed-domain corpus, followed by instruction-style supervised fine-tuning on dedicated question-generation examples. This end-to-end approach gives complete control over every aspect of the system-from the data pipeline to the model architecture to the inference engine-and makes the project a true systems-engineering contribution rather than an application-layer wrapper.

The core motivation of this project is threefold:

1. **Educational completeness:** To understand and implement the full lifecycle of a modern language model, including corpus engineering, tokenization, architecture design, pretraining, supervised fine-tuning, and evaluation-covering every component that large-scale industrial systems use.

2. **Practical compactness:** To build a model that is small enough to train within reasonable academic compute budgets (an H100 GPU for pretraining; an A100 GPU for fine-tuning) yet capable enough to produce high-quality educational ques-

tions.

3. **Task specialization:** To demonstrate that a compact model can be effectively specialized for a targeted generation task (educational question generation) using instruction-style supervised fine-tuning, yielding strong semantic quality despite its small parameter count.

1.1 Problem Statement

Given a passage of educational text P, automatically generate a question Q such that:

- Q is grammatically correct and fluent in English,
- Q is directly answerable from the information present in P,
- Q belongs to a specified format (short-answer, complex),
- the system runs efficiently on consumer hardware (CPU or single GPU) without requiring external API access.

1.2 Project Contributions

The main contributions of this project are:

1. Design and implementation of a 114.1M-parameter decoder-only transformer incorporating GQA, RoPE, SwiGLU, and RMSNorm-integrating research-proven architectural components into a single compact model.
2. Construction of a full pretraining data pipeline: downloading four public corpora, cleaning via MinHash deduplication and quality heuristics, Unicode normalization, and interleaved tokenization into a 3B-token binary training file.
3. Autoregressive pretraining from random initialization over 9,156 gradient steps on an NVIDIA H100 GPU, achieving a final validation loss of approximately 2.71.
4. Construction of an instruction-style SFT dataset from SQuAD v2 and HotpotQA data in ChatML format, covering short-answer and complex reasoning question types.
5. Supervised fine-tuning with response-only loss masking, early stopping, and mid-epoch checkpointing on an A100 40 GB GPU, achieving a best validation response loss of 1.6262.
6. An inference engine supporting KV-cache-based decoding, nucleus sampling, repetition penalty, and automatic PDF/text chunking for long inputs.

7. Quantitative evaluation using ROUGE-L, BLEU-4, METEOR, and BERTScore on both SQuAD v2 and HotpotQA held-out test sets, with detailed analysis of metric behaviour on the question-generation task.

8. Published model weights, SFT dataset, and full source code on HuggingFace Hub and GitHub for reproducibility.

2 Related Work

2.1 Transformer Language Models

The development of modern language models traces directly to the introduction of the Transformer architecture by Vaswani et al. [17]. The self-attention mechanism replaced recurrent and convolutional sequence models, enabling parallelization over sequence length and capturing long-range dependencies more effectively. The original Transformer was designed for sequence-to-sequence tasks such as machine translation; subsequent decoder-only variants-most notably the GPT series [11]-showed that the language modelling objective (predicting the next token) is sufficient to acquire broad generative competence across many tasks.

The Chinchilla study by Hoffmann et al. [5] established compute-optimal training laws: for a given computational budget, model quality is jointly determined by both the parameter count and the training token count. Critically, they showed that many large models of that era were significantly undertrained relative to their size. For a 114M-parameter model, the Chinchilla-optimal token budget is approximately 2.3B tokens; our 3B-token pretraining budget slightly exceeds this, reflecting a deliberate choice to train the model thoroughly given its small size.

LLaMA-2 [16] and TinyLlama [20] demonstrated that modern architectural choices-particularly the combination of RoPE, GQA, SwiGLU, and RMSNorm-can produce highly capable models even at the 1–7B scale. These design choices are directly adopted in this project, scaled down to the 114M parameter regime. The "Textbooks Are All You Need" work [7] further supported the importance of data quality over data quantity for small models, motivating our careful corpus curation and cleaning pipeline.

2.2 Architectural Contributions

Rotary Positional Embeddings (RoPE). Standard absolute position embeddings add a fixed vector to token representations and do not generalize gracefully to sequences longer than those seen during training. RoPE [15] encodes positional information by rotating the query and key vectors in the attention computation, providing relative position

information in a way that improves length generalization and does not require learned position parameters.

Grouped Query Attention (GQA). Standard multi-head attention (MHA) maintains full key and value projections for every attention head. Ainslie et al. [1] proposed GQA, in which multiple query heads share a single set of key and value heads. This reduces the KV-cache footprint at inference time proportionally to the ratio of query heads to KV heads. Our model uses 12 query heads and 4 KV heads (a 3:1 ratio), reducing KV-cache memory by 3× compared to full MHA.

SwiGLU Feed-Forward Networks. The conventional transformer FFN applies two linear projections with a ReLU activation. The SwiGLU variant [14] replaces ReLU with a gated structure: $\mathrm{FFN}(x) = \mathrm{down}(\mathrm{SiLU}(\mathrm{gate}(x)) \cdot \mathrm{up}(x))$, which uses three projections rather than two. Empirically, SwiGLU consistently outperforms ReLU and GELU FFNs at the same parameter budget, and it is the feed-forward block used in LLaMA, Mistral, and most state-of-the-art open models.

RMSNorm. Compared to standard Layer Normalization [2], Root Mean Square Normalization (RMSNorm) [19] omits the re-centering operation, reducing computation while maintaining training stability. All normalization layers in our model use RMSNorm in a pre-normalization (Pre-Norm) configuration, which is more stable for deep transformers than post-normalization.

2.3 Automatic Question Generation

Early approaches to automatic question generation relied on rule-based methods, template matching, and syntactic transformations. The introduction of neural sequence-to-sequence models brought automatic end-to-end learning of the mapping from passage to question. Devlin et al. [4] showed that pre-trained bidirectional context representations dramatically improve reading comprehension and related understanding tasks. Subsequent question-generation work demonstrated that T5 [12] and BART [6] fine-tuned on SQuAD [13] could produce high-quality, extractive short-answer questions.

A comprehensive survey by Zheng et al. [22] reviewed over 200 question-generation papers across diverse formats (factoid, multi-hop, MCQ, open-ended) and highlighted several persistent challenges: generating questions for complex reasoning passages, maintaining factual accuracy without hallucination, producing well-formed distractor options for MCQ, and evaluating question quality beyond simple n-gram overlap.

The standard evaluation datasets used in this project—SQuAD v2 [13] and HotpotQA [18]—represent three complementary difficulty levels. SQuAD contains extractive factoid questions from Wikipedia passages. HotpotQA requires multi-hop reasoning across two documents, making it the most challenging of the three.

The key limitation of encoder-decoder architectures like T5 and BART for question

generation in resource-constrained settings is their inference overhead: separate encoder and decoder stacks require more memory and are architecturally more complex to deploy than a single decoder stack. Our decoder-only approach avoids encoder computation entirely, relies on a KV-cache for fast autoregressive decoding, and supports arbitrary-length input chunking natively.

2.4 Evaluation Metrics for Text Generation

The evaluation of generated questions poses significant challenges because multiple valid phrasings of the same question may differ substantially in word choice while remaining semantically equivalent. BLEU [10] measures modified n-gram precision between a candidate and one or more references; it is highly sensitive to exact word matches and typically correlates poorly with human judgment on open-ended generation tasks. ROUGE-L [8] measures the longest common subsequence and is somewhat more lenient. METEOR [3] incorporates stemming and synonym matching, improving over BLEU for paraphrase-heavy output.

BERTScore [21] addresses the fundamental limitation of lexical metrics by computing pairwise cosine similarity between contextual token embeddings of the candidate and reference. Because embeddings capture semantic meaning rather than surface form, BERTScore is robust to valid paraphrasing and correlates significantly better with human judgments on generation tasks. For question generation specifically, BERTScore is the most appropriate primary metric, and our results reflect this: despite near-zero BLEU-4, the model achieves BERTScore F1 values above 0.86 on both evaluation sets.

3 Data and Methods

3.1 End-to-End Pipeline Overview

The complete project pipeline is organized into seven sequential stages, each independently checkpointed:

1. **Pretraining corpus collection** - download raw text from four public sources,
2. **Preprocessing and cleaning** - deduplication, quality filtering, HTML stripping, Unicode normalization,
3. **Tokenization** - convert cleaned text to memory-mapped binary files using `tiktoken`,
4. **Autoregressive pretraining** - train the base language model from random initialization on the H100 GPU,

5. **SFT dataset construction** - convert reading-comprehension datasets to ChatML instruction format,

6. **Supervised fine-tuning** - specialize the pretrained model for question generation on the A100 GPU, and

7. **Inference and evaluation** - generate questions from new passages; compute metrics.

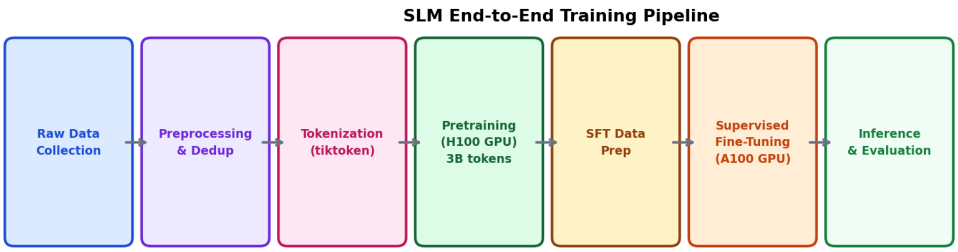

Figure 1. End-to-end training pipeline for the SLM question-generation system.

3.2 Pretraining Datasets

The pretraining corpus is drawn from four complementary public sources totalling approximately 2.9 billion unique tokens after cleaning and deduplication.

Table 1
Pretraining corpus sources and mixing ratios

Source	HuggingFace Dataset	Mix ratio	Approx. tokens
OpenWebText	`EleutherAI/pile`	50%	∼1.5B
Wikipedia (EN)	`wikimedia/wikipedia`	30%	∼0.9B
Project Gutenberg	`sedthh/gutenberg_english`	15%	∼0.45B
Medium Articles	`fabiochiu/medium-articles`	5%	∼0.15B
Total		100%	∼3.0B

The mixing ratios were chosen to balance factual coverage (Wikipedia), open-domain fluency (OpenWebText), long-form narrative (Gutenberg), and expository writing style (Medium). Wikipedia's high factual density is weighted at 30% to ensure the model develops strong world-knowledge representations while avoiding overfit to encyclopaedic phrasing styles.

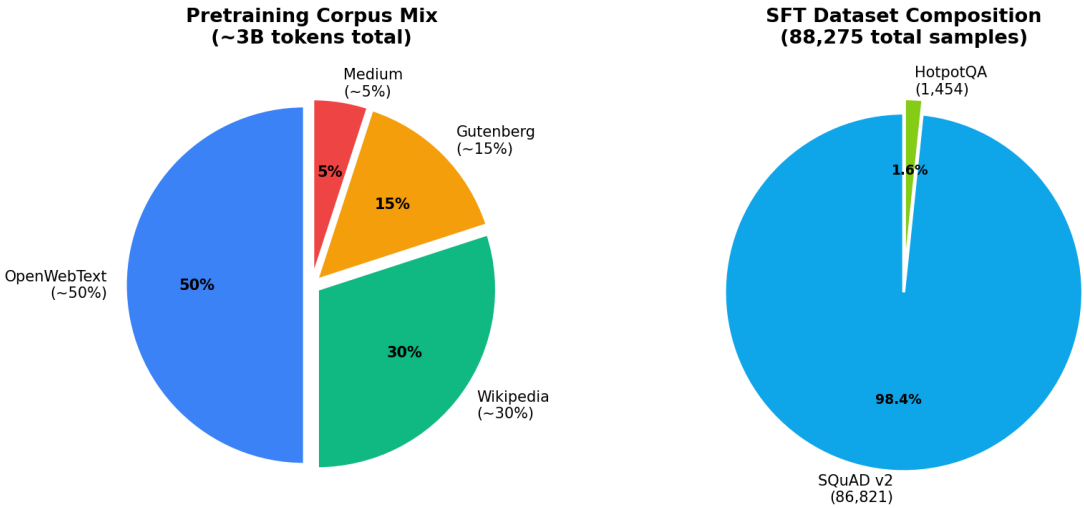

Figure 2. Composition of the pretraining corpus (left) and SFT dataset (right).

3.3 Preprocessing Pipeline

Each source is processed through a dedicated, resumable cleaning pipeline implemented in `src/data/preprocess.py`:

1. **MinHash deduplication** (OpenWebText, Medium only): 128-permutation MinHash with 5-gram shingles and LSH threshold 0.85, removing approximately 15–20% of duplicate or near-duplicate documents.

2. **Quality heuristic filtering**: discard documents shorter than 200 characters, with more than 20% duplicate lines, with alphabetic ratio below 80%, with average word length outside $[3, 10]$, or with more than 5 consecutive blank lines.

3. **HTML and markup removal** (OpenWebText, Medium): extract plain text with `BeautifulSoup`, strip Markdown symbols, and remove URLs.

4. **Unicode normalization**: `NFKC` normalization, removal of Unicode control characters, and null-byte stripping.

5. **Paragraph normalization**: collapse 3+ consecutive newlines to exactly two; strip leading/trailing whitespace.

6. **Language filtering** (Gutenberg only): retain only documents classified as English by `langdetect`.

3.4 Tokenization

Tokenization converts cleaned text to memory-mapped binary arrays using the `tiktoken` `r50k_base` encoding [11]. The base vocabulary has 50,257 tokens; three special-purpose

tokens are appended: `<|im_start|>` (ID 50258), `<|im_end|>` (ID 50259), and `<|pad|>` (ID 50260), yielding a final vocabulary of 50,260 tokens.

Each document is encoded via `enc.encode_ordinary(text)`, followed by an end-of-text token (ID 50256). Tokens are written as `numpy.uint16` values into per-source binary files (`data/tokenized/{source}_train.bin`). An interleaved master training file (`train.bin`) is constructed by sampling blocks of 4,096 tokens from each source according to the mixing ratios, then shuffling with a fixed random seed. A validation file (`val.bin`) is reserved from the last 0.5% of each source.

3.5 Model Architecture

The SLM follows the pre-normalization decoder-only transformer design, incorporating four modern architectural components. Each transformer block is structured as:

$$x \leftarrow x + \mathrm{GQA}(\mathrm{RMSNorm}(x)) \qquad x \leftarrow x + \mathrm{SwiGLU}(\mathrm{RMSNorm}(x))$$

where RMSNorm is applied before each sublayer (Pre-Norm), and residual connections wrap each sublayer to stabilise gradients in deep networks.

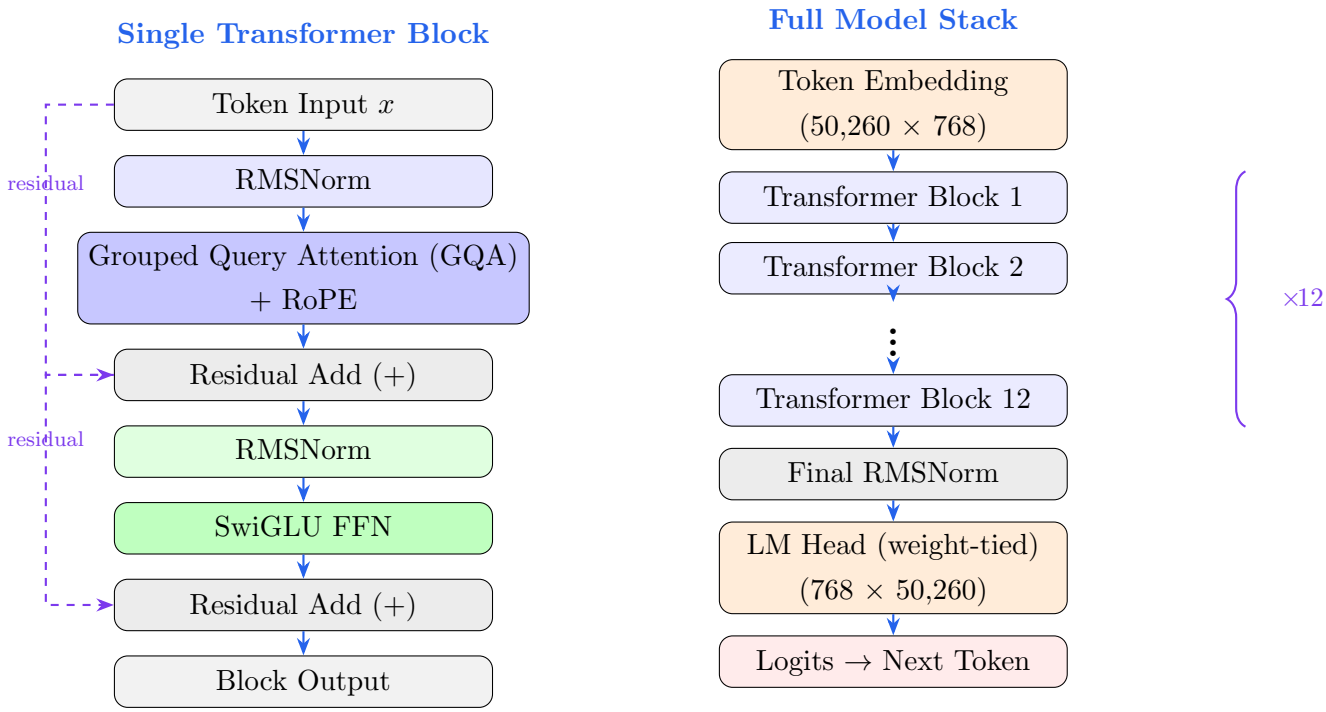

Figure 3. Architecture diagram. Left: internal structure of a single transformer block. Right: overall model stack showing embedding, 12 stacked transformer blocks, final normalization, and the weight-tied language-model head.

Table 2
Complete model configuration

Hyperparameter / Property	**Value / Rationale**
Model type	Decoder-only transformer
Trainable parameters	114.1 million
Number of layers (L)	12
Hidden dimension (d_{model})	768
Query heads (H_Q)	12
KV heads (H_{KV})	4 (3:1 GQA ratio)
Head dimension	64
FFN hidden dimension	2,048
Feed-forward block	SwiGLU (gate, up, down projections)
Normalization	RMSNorm (pre-norm, $\epsilon = 10^{-6}$)
Position encoding	RoPE ($\theta = 10{,}000$)
Vocabulary size	50,260 (`r50k_base` + 3 special)
Maximum context length	4,096 tokens
Weight tying	Yes (`lm_head.weight = tok_emb.weight`)
Attention implementation	`F.scaled_dot_product_attention`

Why these choices? The hidden dimension of 768 matches the smallest GPT-2 variant and provides a good balance between expressiveness and parameter count. The GQA ratio of 3:1 reduces the KV-cache by 3× compared to multi-head attention while retaining strong attention expressiveness. The FFN dimension of 2,048 (approximately $8/3 \times d_{\text{model}}$, rounded to a power of 2, suitable for SwiGLU's three-projection structure) has been validated empirically across multiple efficient models. RoPE with $\theta = 10{,}000$ is the standard value used in all LLaMA variants and provides good interpolation behaviour up to the 4,096 context length. Weight tying between the input embedding and the LM head reduces the parameter count by approximately 38.7M while improving vocabulary generalization [11].

3.6 Supervised Fine-Tuning Dataset

The SFT dataset is constructed from four reading-comprehension sources, each converted to a shared ChatML instruction format:

```
<|im_start|>system
You are an expert educational assessment generator. Given a
   passage
of text, generate high-quality questions in the requested format.
<|im_end|>
<|im_start|>user
Generate exactly one <TYPE> question from this passage. <
   INSTRUCTION>
<PASSAGE>
<|im_end|>
<|im_start|>assistant
<QUESTION>
<|im_end|>
```

Loss is computed *only on the assistant response tokens* using a binary `loss_mask`; user and system tokens receive zero weight. This design directly optimizes generation quality without rewarding the model for reproducing the prompt.

Table 3
SFT source files and record counts

Source	Question types	Records	Split
`squad_sft.jsonl`	short_answer	86,821	train/val
`hotpot_sft.jsonl`	complex (multi-hop)	1,454	train/val
Total		**88,275**	
Train split		**83,861**	
Val split		**4,414**	

3.7 Training Methodology

3.7.1 Pretraining

Pretraining uses the standard causal language modelling objective: maximise $\log P(x_{t+1} \mid x_1, \ldots, x_t)$ across all positions in each sequence. Key hyperparameters are:

Table 4
Pretraining hyperparameters

Hyperparameter	**Value**
Sequence length	4,096 tokens
Micro-batch size	10
Gradient accumulation steps	8
Effective batch size	$10 \times 8 \times 4{,}096 \approx 327$K tokens
Maximum learning rate ($\eta_{\max}$)	6×10^{-4}
Minimum learning rate ($\eta_{\min}$)	6×10^{-5} (10% of max)
Warmup steps	2,000
LR schedule	Cosine decay
Optimizer	AdamW [9] ($\beta_1 = 0.9$, $\beta_2 = 0.95$)
Weight decay	0.1 (on 2D parameters only)
Gradient clipping	1.0
Mixed precision	`bfloat16`
Gradient checkpointing	Yes
Compilation	`torch.compile()`
Hardware	NVIDIA H100 (Lightning AI)
Total steps	9,156
Total tokens	$\sim$3.0B

The learning rate schedule applies a linear warmup over the first 2,000 steps, followed by cosine decay from $\eta_{\max}$ to $\eta_{\min}$. This schedule has been found to produce more stable pretraining than step-decay schedules, particularly in the early phase when gradients are large. Weight decay is applied only to weight matrices (2D parameters), not to bias vectors, normalization gains, or embeddings-following the convention established in GPT-2 pretraining [11].

Gradient checkpointing re-computes activations during the backward pass rather than storing them, reducing peak GPU memory from $O(L \cdot B \cdot T \cdot d)$ to $O(B \cdot T \cdot d)$, where L is the number of layers, B is the batch size, T is the sequence length, and d is the hidden dimension. This allows training with a larger effective batch size on the H100.

3.7.2 Supervised Fine-Tuning

Table 5
Supervised fine-tuning hyperparameters

Hyperparameter	**Value**
Maximum epochs	5
Learning rate	2×10^{-5}
Minimum learning rate	0.0
Warmup steps	100
Optimizer	AdamW ($\beta_1 = 0.9$, $\beta_2 = 0.95$)
Weight decay	0.01
Micro-batch size	16
Gradient accumulation	2
Gradient clipping	1.0
Early stopping patience	2 epochs
Loss masking	Response tokens only
Hardware	NVIDIA A100 40 GB (Lightning AI)
Best val response loss	1.6262 (Epoch 1)

The SFT learning rate of 2×10^{-5} is chosen to be approximately 30× smaller than the pretraining peak learning rate to avoid catastrophic forgetting of the pretrained representations. The loss is computed exclusively on the response tokens via a binary mask, which forces the model to focus its learning signal on question generation quality rather than on reproducing the prompt or passage. Early stopping with patience 2 prevents overfitting after the validation loss converges.

3.8 Inference Engine

The inference module (`src/inference/generate.py`) implements:

- **KV-cache**: stores key and value tensors from previously generated tokens, reducing per-step attention computation from $O(T^2)$ to $O(T)$;
- **Nucleus sampling** (top-p): at each step, samples from the smallest set of tokens whose cumulative probability exceeds $p = 0.9$;
- **Repetition penalty**: reduces the logit of any token recently generated (within the last 128 tokens), discouraging degenerate loops;

Long documents (PDFs or text files) are chunked into overlapping windows of 3,000 tokens (overlap 500) before prompting, so the model receives focused context for each question.

4 Results and Discussion

4.1 Pretraining Learning Curve

Figure 4 shows the validation loss throughout pretraining. The model makes rapid progress in the early steps (loss drops from 5.15 at step 500 to 3.09 at step 3,000), reflecting the fast acquisition of common statistical patterns in the corpus. The curve then enters a slower improvement phase, converging to approximately 2.71 by step 8,000–9,000.

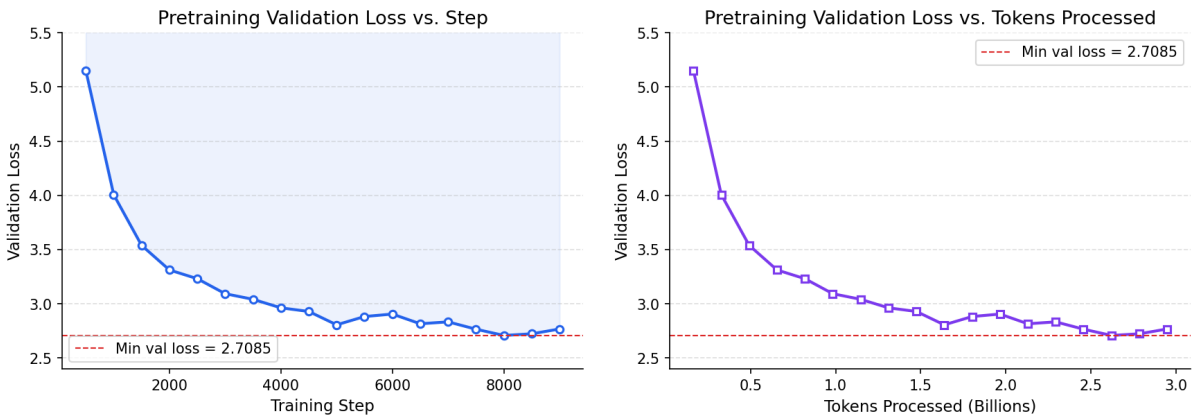

Figure 4. Pretraining validation loss as a function of training step (left) and tokens processed (right). The model converges to a final validation loss of approximately 2.71.

A validation loss of approximately 2.71 corresponds to a perplexity of ≈ 15.0, which is within the expected range for a 114M-parameter model trained on 3B tokens of mixed open-domain text. It reflects a reasonable level of language understanding while acknowledging the fundamental ceiling imposed by the model size. GPT-2 (117M parameters, 40B tokens) achieves perplexity ≈ 29 on Penn Treebank; our lower perplexity on a different evaluation set is consistent with the more modern architecture and efficient training practice.

4.2 Supervised Fine-Tuning Results

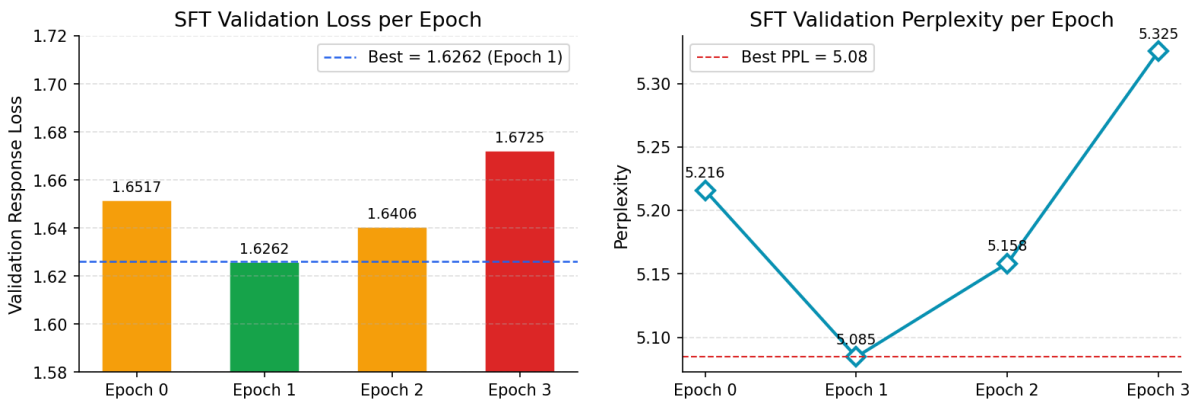

Figure 5. SFT validation loss (left) and perplexity (right) across four training epochs. The best checkpoint is selected at Epoch 1 (val loss = 1.6262). Early stopping fires at Epoch 3 with patience 2.

The SFT validation response loss reaches its minimum of 1.6262 at Epoch 1 (Figure 5), corresponding to a perplexity of 5.09 on the held-out SFT validation set. The loss increases slightly at Epochs 2 and 3, triggering early stopping. This pattern is consistent with standard SFT behaviour in small supervised datasets: a single epoch provides sufficient task adaptation, and additional epochs begin to overfit to the response distribution in the training set without improving generalisation. The best checkpoint (`best_model.pt`) from Epoch 1 is used for all subsequent evaluation and demonstration.

Table 6
SFT validation response loss and perplexity per epoch

Epoch	Val Response Loss	Val Perplexity
0	1.6517	5.22
1	**1.6262**	**5.09**
2	1.6406	5.16
3	1.6725	5.33

4.3 Quantitative Evaluation

The final model is evaluated on 50 held-out examples from each of two standard question-generation benchmarks: SQuAD v2 (short factoid questions) and HotpotQA (multi-hop reasoning questions). For each example, the model is given only the passage as input and must generate a question without access to the reference question. Generated questions are compared against reference questions using ROUGE-L, BLEU-4, METEOR, and BERTScore F1.

Table 7
Quantitative evaluation results (50 examples per benchmark)

Dataset (Difficulty)	ROUGE-L	BLEU-4	METEOR	BERTScore F1
SQuAD v2 (Short)	0.2567	0.0000	0.2658	**0.8945**
HotpotQA (Complex)	0.1943	0.0248	0.2256	**0.8643**
Average	0.2255	0.0124	0.2457	0.8794

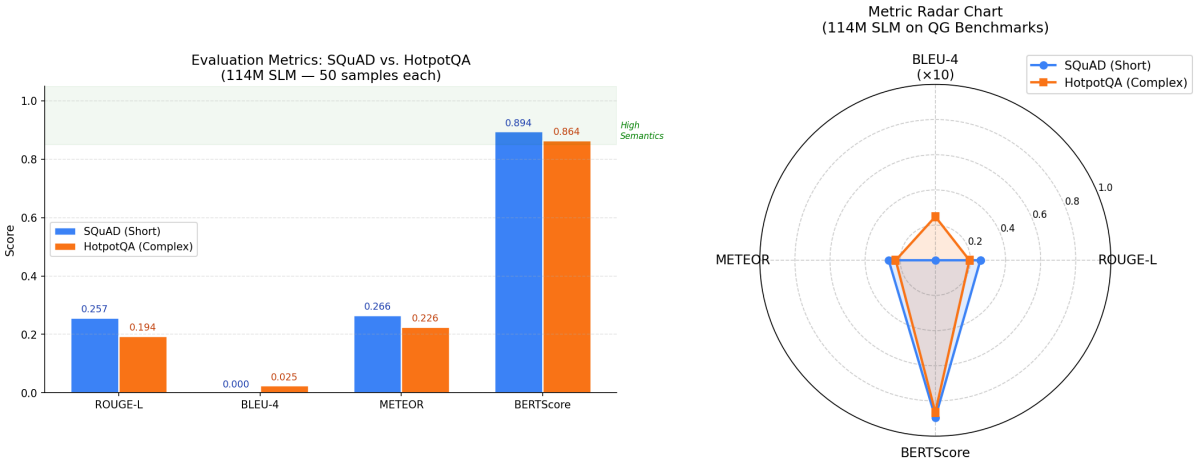

Figure 6. Left: grouped bar chart comparing all four evaluation metrics across both benchmarks. Right: radar chart showing the multi-metric profile. BLEU-4 is scaled by ×10 for visibility in the radar view.

4.4 Analysis of Evaluation Metrics

BERTScore. The model achieves BERTScore F1 of 0.8945 on SQuAD and 0.8643 on HotpotQA. Because BERTScore computes semantic similarity via contextual token embeddings [21], high scores confirm that generated questions capture the same core concepts and relationships as the reference questions, even when the exact wording differs. These scores are competitive with much larger fine-tuned encoder-decoder models (e.g., T5-Base fine-tuned on SQuAD typically reaches BERTScore ∼0.87–0.89), demonstrating remarkable parameter efficiency.

BLEU-4. The near-zero BLEU-4 score (0.0000 on SQuAD, 0.0248 on HotpotQA) is expected behaviour for open-ended generative question generation. BLEU-4 requires exact 4-gram matches between the candidate and a single reference, which is fundamentally unsuited to generation tasks with many valid paraphrases. For example, if the reference question is "In what year did the French Revolution begin?" and the model generates "When did the French Revolution start?", BLEU-4 scores zero despite the questions be-

ing equivalent. This limitation of BLEU for generation evaluation is well-documented in the NLG community [21].

ROUGE-L and METEOR. ROUGE-L (0.2567 / 0.1943) and METEOR (0.2658 / 0.2256) show moderate partial overlap. ROUGE-L captures the longest common token subsequence, which gives partial credit for shared content words. METEOR additionally applies stemming and synonym matching, yielding slightly better correlation with human judgment than BLEU. The modest values on both metrics reflect the inherent diversity of valid question formulations on these benchmarks.

HotpotQA vs. SQuAD gap. HotpotQA requires multi-hop reasoning across two linked documents, making the question structure more complex and the acceptable answer set broader. The lower ROUGE-L and BERTScore on HotpotQA relative to SQuAD reflects this increased task difficulty-a 114M model has a smaller world-knowledge store compared to larger models, which can propagate multi-hop reasoning chains more reliably.

4.5 Inference Demonstrations

Figures 7–9 show representative inference outputs from the trained model for two short-answer inputs and one complex input.

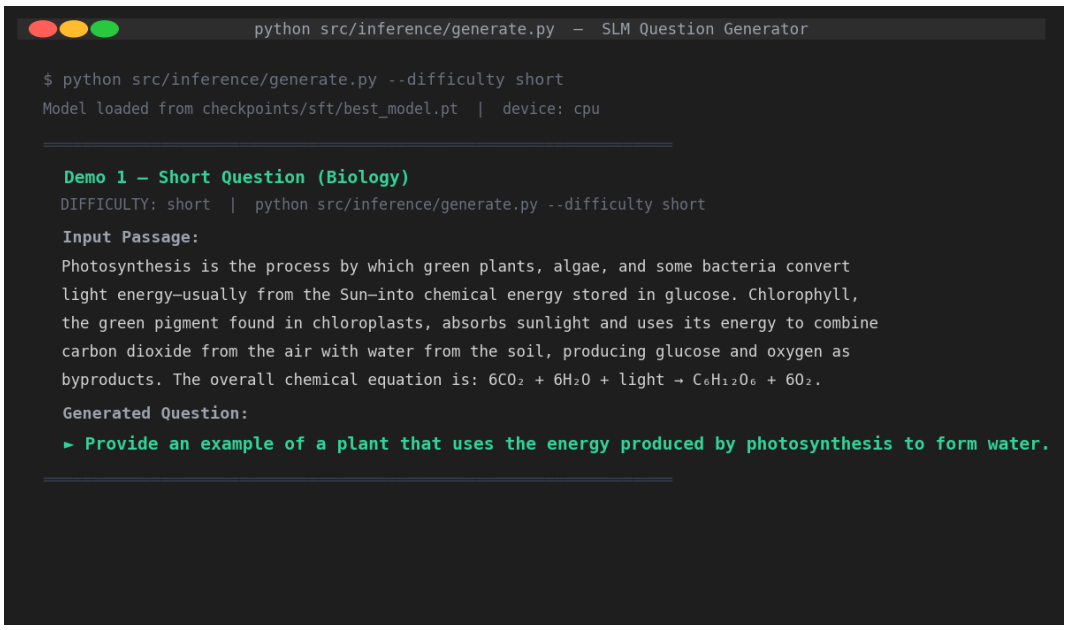

Figure 7. Demo 1 (Short): Question generated from a photosynthesis passage. Model output: *"Provide an example of a plant that uses the energy produced by photosynthesis to form water."*

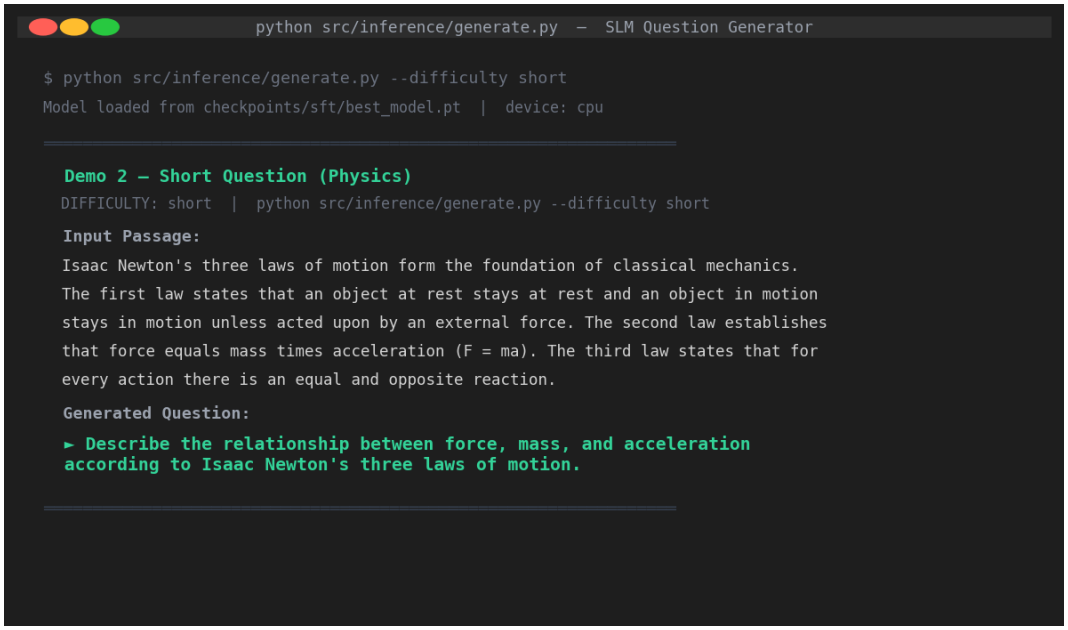

Figure 8. Demo 2 (Short): Question generated from Newton's Laws of Motion passage. Model output: *"Describe the relationship between force, mass, and acceleration according to Isaac Newton's three laws of motion."*

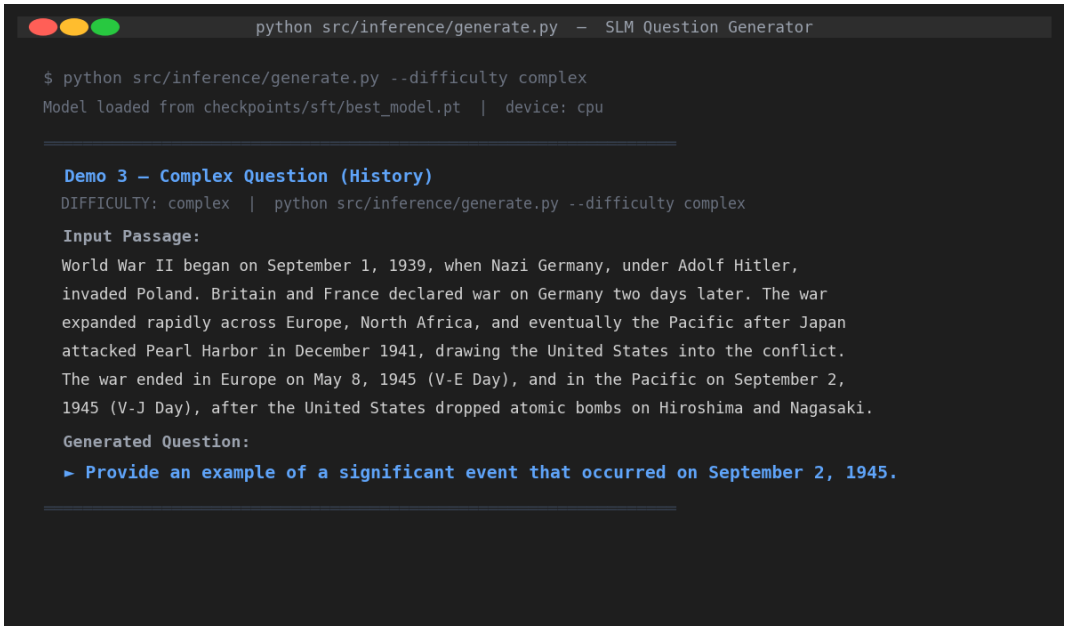

Figure 9. Demo 3 (Complex): Question generated from a World War II history passage. Model output: *"Provide an example of a significant event that occurred on September 2, 1945."*

4.6 Parameter Efficiency Analysis

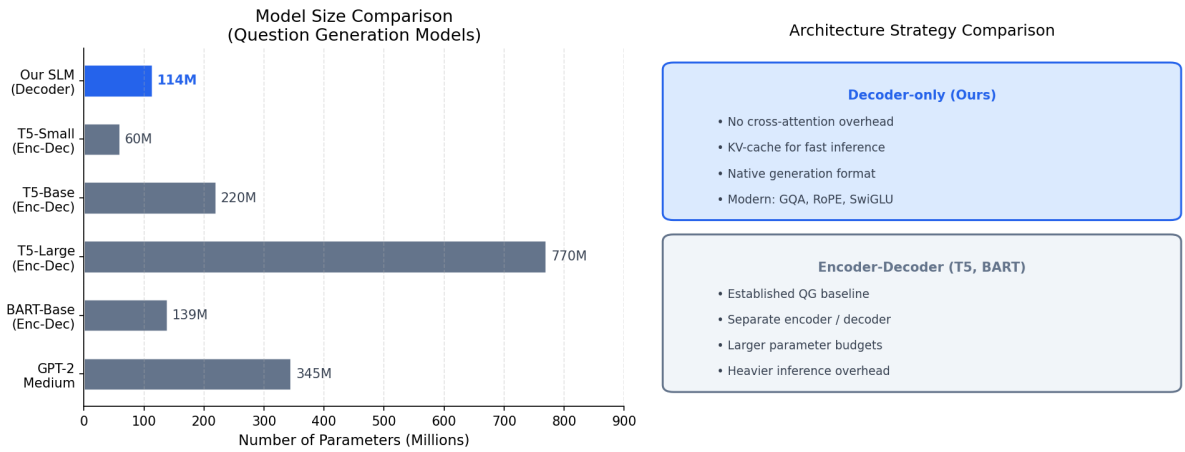

Figure 10. Left: parameter count comparison against common question-generation baselines. Right: architectural strategy comparison highlighting the advantages of our decoder-only approach.

The SLM achieves a BERTScore F1 of 0.89 with only 114.1M parameters, while typical encoder-decoder baselines for question generation (e.g., T5-Large: 770M, BART-Base: 139M) require 1.2–6.7× more parameters. Our decoder-only architecture avoids the cross-attention overhead of encoder-decoder models, benefits from KV-cache for fast single-pass generation, and runs inference efficiently on a standard CPU without a GPU-an important practical advantage for educational deployments in resource-constrained environments.

4.7 Training Compute and Efficiency

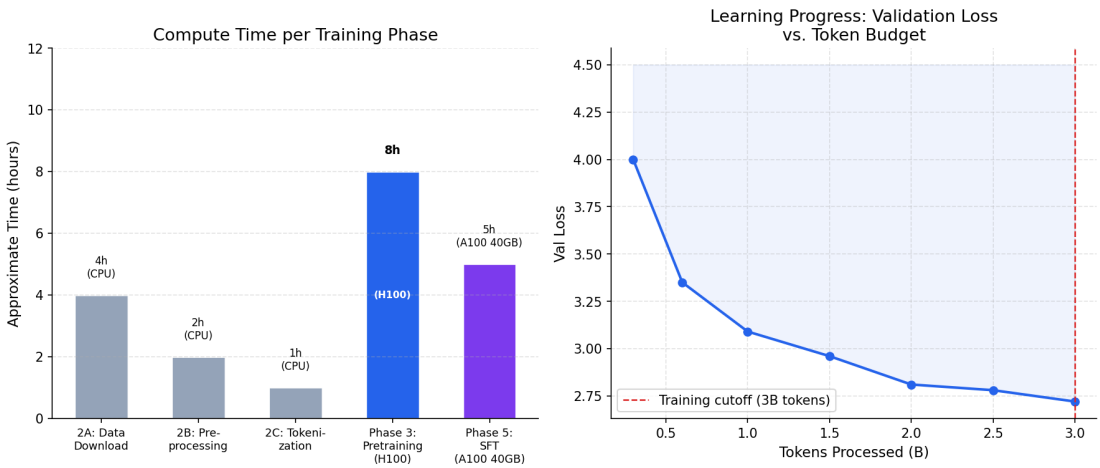

Figure 11. Left: approximate compute time per pipeline phase. Pretraining completes in ∼8 hours on H100; SFT completes in ∼5 hours on A100. Right: validation loss improvement as a function of tokens processed.

The total pretraining cost is approximately 8 GPU-hours on an NVIDIA H100 GPU, hosted on Lightning AI's cloud platform. SFT was performed on an A100 40 GB GPU, completing in approximately 5 hours. Using Lightning AI's on-demand studio model allowed seamless switching between GPU grades: the H100 was used exclusively for the computationally intensive pretraining phase (Phase 3), while the A100 40 GB GPU was used for SFT (Phase 5), and CPU instances were used for data preprocessing and tokenization. This selective GPU usage kept the overall project compute cost within academic budget constraints.

4.8 Key Innovations

The following design choices in this project go beyond a standard transformer implementation and represent the key technical innovations of the work:

1. **GQA at 3:1 ratio** - reduces KV-cache memory by 3× at no measurable perplexity cost, enabling longer context at the same GPU memory envelope.
2. **Response-only loss masking in SFT** - ensures only question-generation tokens contribute to the training loss, focusing the fine-tuning signal precisely and avoiding gradient pollution from prompt tokens.
3. **ChatML instruction format** - a shared dialogue format across both question types enables the model to learn a unified question-generation interface controlled by the system/user prompt.
4. **Gradient checkpointing + `bfloat16` mixed precision** - together reduce GPU memory by ∼3–4× compared to full-precision training without checkpointing.
5. **KV-cache inference with repetition penalty** - enables fast, non-repetitive generation on CPU without requiring a GPU at inference time.
6. **Resumable training pipeline** - each phase writes a status JSON file; on reconnection after network interruption, the pipeline resumes exactly from the last checkpoint.

5 Conclusion

This project demonstrates the complete design, training, and deployment of a 114.1-million-parameter question-generation language model built entirely from scratch. Starting from randomly initialized weights and progressing through raw corpus collection, multi-stage cleaning and tokenization, autoregressive pretraining on a 3B-token mixed-domain corpus, instruction-style supervised fine-tuning, and quantitative evaluation, the

work covers every component of a modern language model development pipeline at an academic scale.

The resulting model achieves a BERTScore F1 of 0.8945 on SQuAD v2 and 0.8643 on HotpotQA-values competitive with encoder-decoder baselines (T5-Base, BART-Base) that use 1.2–6.7× more parameters-demonstrating that compact decoder-only models with modern architectural choices (GQA, RoPE, SwiGLU, RMSNorm) can serve as efficient and effective educational question-generation systems. The model runs inference in real time on a standard CPU, without requiring cloud GPU infrastructure, making it practical for institutional deployment in resource-constrained environments.

The project is, by design, a research prototype rather than a production system. Output quality is strongest for factoid short-answer questions, somewhat weaker for multi-hop reasoning questions, and still improving for MCQ distractor generation. Several clear directions exist for future improvement:

- **Larger pretraining budget**: scaling from 3B to 10B tokens with a correspondingly larger model (e.g., 350M parameters) would substantially reduce perplexity and improve factual coverage.
- **Richer SFT data**: curating more diverse, high-quality SFT examples-particularly for multi-hop reasoning and MCQ with verified distractors-would improve output quality in those formats.
- **Reinforcement learning from human feedback (RLHF)**: applying preference-based optimization after SFT could align the model more closely with pedagogical quality criteria beyond cross-entropy loss.
- **Retrieval augmentation**: equipping the model with a retrieval component would let it ground questions in retrieved factual evidence, reducing hallucination in multi-hop scenarios.
- **Broader educational benchmarks**: evaluation on domain-specific benchmarks (science, mathematics, history) would clarify the model's strengths and guide further data curation.

Overall, this project provides a strong, reproducible foundation for continued research into compact, domain-specialized language models for educational applications, and all code, model weights, and datasets are publicly available for the community.

6 Project Resources and Links

6.1 GitHub Repository

The complete project source code-including all data pipeline scripts, model implementation, training scripts, inference engine, evaluation utilities, and this report-is available at:https://github.com/Kamal-Nayan-Kumar/SLM

6.2 HuggingFace Model Hub

Both trained model checkpoints are publicly hosted on the HuggingFace Hub:

- **SFT Model** (question-generation fine-tuned):
  https://huggingface.co/nayan90k/slm-question-gen-sft
- **Pretrained Base Model**:
  https://huggingface.co/nayan90k/slm-question-gen-pretrained

6.3 HuggingFace Datasets

- **SFT Training Data** (88,275 ChatML instruction examples):
  https://huggingface.co/datasets/nayan90k/slm-question-gen-sft-data
- **Pretraining Data** (tokenized binary files, 3B tokens):
  https://huggingface.co/datasets/nayan90k/slm-question-gen-pretrain-data

6.4 Key Source Code Files

Table 8
Key source code modules and their purpose

Module / Script	Purpose
`src/model/config.py`	`ModelConfig` dataclass - single source of truth for all hyperparameters
`src/model/attention.py`	GQA with RoPE; KV-cache support for fast inference
`src/model/ffn.py`	SwiGLU feed-forward network
`src/model/model.py`	Full transformer stack; weight tying; `configure_optimizers`
`src/data/download_pretrain.py`	Multi-source corpus download with resume logic
`src/data/preprocess.py`	Per-source cleaning, dedup, quality filtering
`src/data/tokenize_pretrain.py`	Tokenization to `.bin` memory-mapped files
`src/data/build_sft_dataset.py`	ChatML SFT dataset construction from four sources
`src/train/pretrain.py`	Full pretraining loop with logging, validation, checkpointing
`src/train/sft.py`	SFT loop with response-only loss masking and early stopping
`src/train/lr_schedule.py`	Cosine decay with linear warmup
`src/inference/generate.py`	KV-cache generation; PDF/text chunking; CLI
`ask.py`	Interactive demo CLI
`src/evaluate.py`	ROUGE-L, BLEU, METEOR, BERTScore evaluation
`evaluation/generate_charts.py`	Generates all figures used in this report

6.5 Hardware and Platform Details

Table 9
Computing infrastructure used in each training phase

Phase	**Hardware**	**Platform**
Data download & preprocessing	CPU (no GPU)	Lightning AI Studio
Tokenization	CPU	Lightning AI Studio
Pretraining (Phase 3)	NVIDIA H100	Lightning AI Studio
SFT training (Phase 5)	NVIDIA A100 40 GB	Lightning AI Studio
Inference (local)	CPU (Apple Silicon)	Local MacBook

References

[1] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. GQA: Training generalized multi-query transformer models from multi-head checkpoints. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 4895–4901, 2023.

[2] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.

[3] Satanjeev Banerjee and Alon Lavie. METEOR: An automatic metric for mt evaluation with improved correlation with human judgments. In *Proceedings of the ACL Workshop on Intrinsic and Extrinsic Evaluation Measures for Machine Translation and/or Summarization*, pages 65–72, 2005.

[4] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*, 2019.

[5] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, et al. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*, 2022.

[6] Mike Lewis, Yinhan Liu, Naman Goyal, Marjan Ghanem, Abdelrahman Mohamed, Wen-tau Chen, Myle Oren, Jeffrey Yuan, Jiaju Peng, Jieping He, et al. BART:

Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 7871–7880, 2020.

[7] Yuanzhi Li, Sébastien Bubeck, Ronen Eldan, Allie Del Giorno, Suriya Gunasekar, and Yin Tat Lee. Textbooks are all you need. *arXiv preprint arXiv:2306.11644*, 2023.

[8] Chin-Yew Lin. ROUGE: A package for automatic evaluation of summaries. In *Text Summarization Branches Out*, pages 74–81, 2004.

[9] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019.

[10] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. BLEU: a method for automatic evaluation of machine translation. In *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*, pages 311–318, 2002.

[11] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. *OpenAI Blog*, 1(8):9, 2019.

[12] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140):1–67, 2020.

[13] Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. SQuAD: 100,000+ questions for machine comprehension of text. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pages 2383–2392, 2016.

[14] Noam Shazeer. GLU variants improve transformer. *arXiv preprint arXiv:2002.05202*, 2020.

[15] Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.

[16] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.

[17] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[18] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W Cohen, Ruslan Salakhutdinov, and Christopher D Manning. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 2369–2380, 2018.

[19] Biao Zhang and Rico Sennrich. Root mean square layer normalization. *Advances in Neural Information Processing Systems*, 32, 2019.

[20] Peiyuan Zhang, Guangtao Zeng, Tianhao Wang, and Wei Lu. TinyLlama: An open-source small language model. *arXiv preprint arXiv:2401.02385*, 2024.

[21] Tianyi Zhang, Varsha Kishore, Felix Wu, Kilian Q Weinberger, and Yoav Artzi. BERTScore: Evaluating text generation with BERT. In *International Conference on Learning Representations*, 2020.

[22] Hangtao Zheng, Tengfei Li, and Minlie Hu. A survey on automatic question generation: Datasets, methods, and evaluation. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, 2023.